

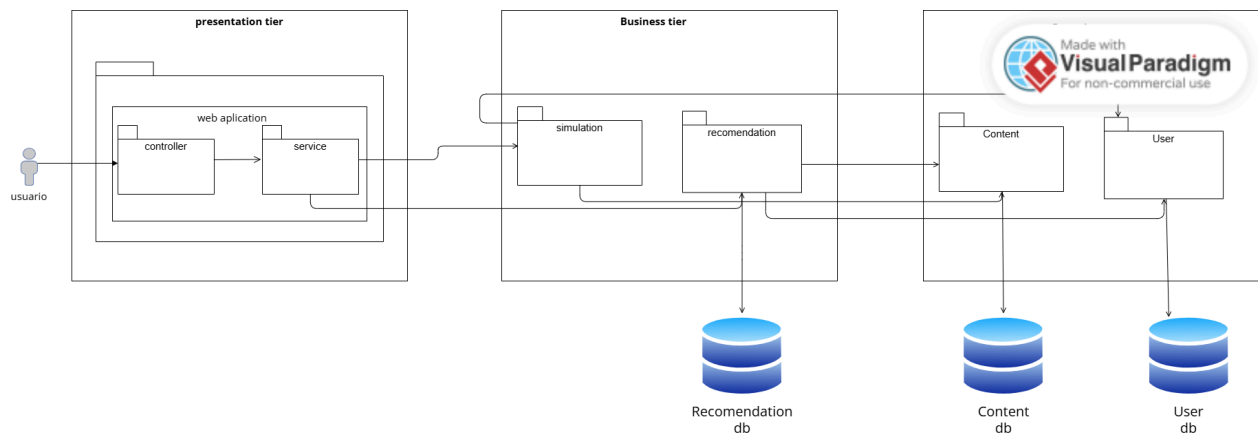
Integrantes:

- Arley Bernal Muñeton
- Juan Camilo Alba Castro
- Laura Garzón Arias
- Carlos Villegas Ruiz

Introducción

En este taller se diseña la arquitectura de una aplicación web para apoyar el aprendizaje de un curso de Estructuras de Datos. El sistema permite gestionar contenidos, simular el avance de un estudiante y generar recomendaciones según su desempeño.

Vista Lógica



Web Application: Esta aplicación web funciona bajo el patrón MVC y desempeña un papel central en la interfaz gráfica y la gestión de solicitudes hacia los servicios del sistema. Está construida con JSF sobre Wildfly y expone las vistas a través de páginas .xhtml. Internamente se compone de Beans (ContentBean, SimulationBean, RecommendationBean, UserBean) que actúan como controladores, y un ServiceClient que actúa como capa de servicio, siendo el único punto de salida hacia los microservicios mediante llamadas HTTP REST.

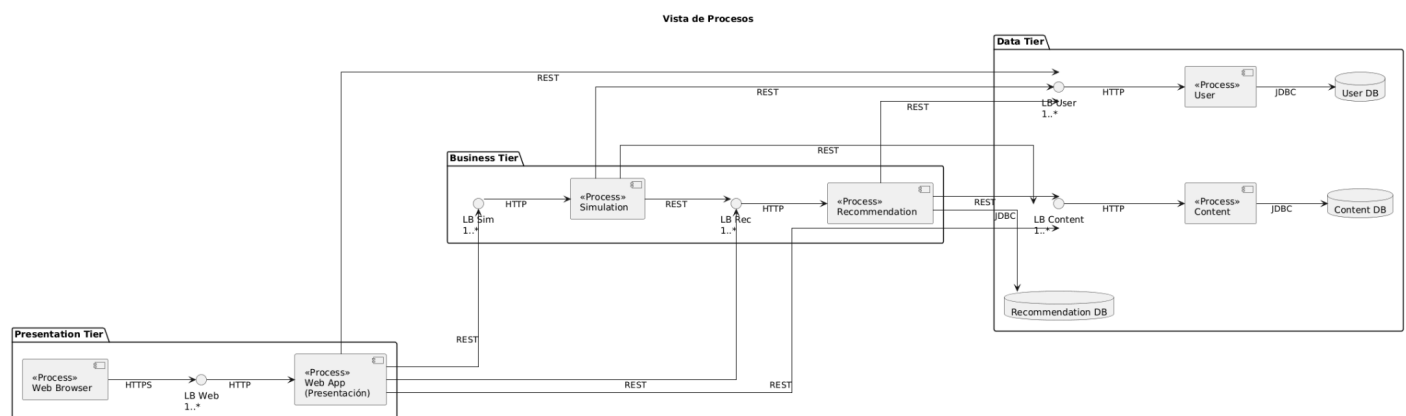
Simulation: Este servicio es responsable de orquestar el flujo completo del curso para un estudiante. Su función principal es recibir un enrollmentId y simular el recorrido del estudiante por todos los módulos, lecciones y contenidos del curso. Para llevar a cabo esta tarea, consume el servicio de Content para obtener la estructura del curso (módulos, lecciones, videos, artículos, archivos y quizzes), consume el servicio de User para registrar el inicio y finalización de cada lección y los intentos de quiz, y al terminar cada lección invoca al servicio de Recommendation para generar el análisis de debilidades del estudiante. Este servicio no tiene base de datos propia, ya que delega toda la persistencia en los servicios de Data Tier.

Recommendation: Este servicio es responsable de analizar el avance del estudiante y generar recomendaciones de refuerzo. Su lógica principal evalúa si el score general de una lección es menor al 70%, considerando tanto el estado de completitud de la lección como el puntaje obtenido en los quizzes. Para realizar este análisis, consume el servicio de User para obtener el progreso de lecciones e intentos de quiz del estudiante, y consume el servicio de Content para obtener los contenidos de refuerzo asociados a cada lección. Las recomendaciones generadas se persisten en su propia base de datos Recommendation DB.

Content: Este servicio se centra en gestionar toda la estructura del contenido del curso. Es responsable de administrar la jerarquía Course → Module → Lesson → Content, donde los contenidos pueden ser de tipo VIDEO, ARTICLE, FILE o QUIZ. También expone un endpoint de refuerzo (/reinforcement) que retorna los contenidos marcados con propósito de refuerzo para una lección dada. Toda la información se persiste en su propia base de datos Content DB mediante repositorios JPA (CourseRepository, LessonRepository, ContentRepository, QuizRepository).

User: Este servicio gestiona toda la información relacionada con el estudiante dentro de la plataforma. Es responsable de administrar las inscripciones al curso (Enrollment), el progreso por lección (LessonProgress) con estados STARTED y COMPLETED, y los intentos de quiz (QuizAttempt) con sus puntajes obtenidos y máximos. Toda esta información se persiste en su propia base de datos User DB, garantizando el aislamiento de datos respecto a los demás servicios.

Vista de procesos



Web Application:

En la vista de procesos, la aplicación web se encarga de gestionar las solicitudes HTTP provenientes de los usuarios finales a través del navegador. Este componente recibe peticiones como GET, POST, PUT y DELETE, con el propósito de entregar recursos educativos como páginas HTML, archivos CSS, scripts JavaScript, imágenes y contenido multimedia. Además, actúa como punto de entrada hacia los servicios de negocio, redirigiendo las solicitudes hacia los módulos correspondientes mediante servicios REST.

Para garantizar alta disponibilidad y escalabilidad, las solicitudes son distribuidas a través de un balanceador de carga.

Simulation:

Este proceso tiene como objetivo simular el avance de un estudiante dentro del curso de Estructuras de Datos. Gestiona la lógica relacionada con el progreso académico, evaluando el estado actual del usuario y determinando el flujo de actividades dentro de la plataforma. Para su funcionamiento, consume información de los servicios de contenido y usuarios, permitiendo generar una experiencia dinámica y adaptativa.

Recommendation:

El módulo de recomendaciones se encarga de sugerir contenido educativo relevante al estudiante, basándose en su progreso y comportamiento dentro de la plataforma. Este proceso analiza la información disponible y propone el siguiente tema, recurso o actividad que el usuario debería abordar. Para ello, interactúa con los servicios de contenido y usuario, y almacena resultados en su propia base de datos para optimizar futuras recomendaciones.

Content:

Este proceso administra todos los recursos educativos del sistema, tales como documentos, videos, imágenes, presentaciones y enlaces externos. Permite la consulta y gestión del contenido almacenado en la base de datos, garantizando su disponibilidad para los demás módulos del sistema. La interacción con la base de datos se realiza mediante conexiones JDBC.

User:

El proceso de usuario se encarga de la gestión de la información relacionada con los estudiantes, incluyendo datos personales, progreso, historial y configuraciones. Este módulo es fundamental para personalizar la experiencia del usuario dentro de la plataforma. Al igual que el proceso de contenido, accede a su base de datos mediante JDBC.

Bases de Datos:

El sistema cuenta con múltiples bases de datos especializadas:

Content DB: almacena los recursos educativos del curso.

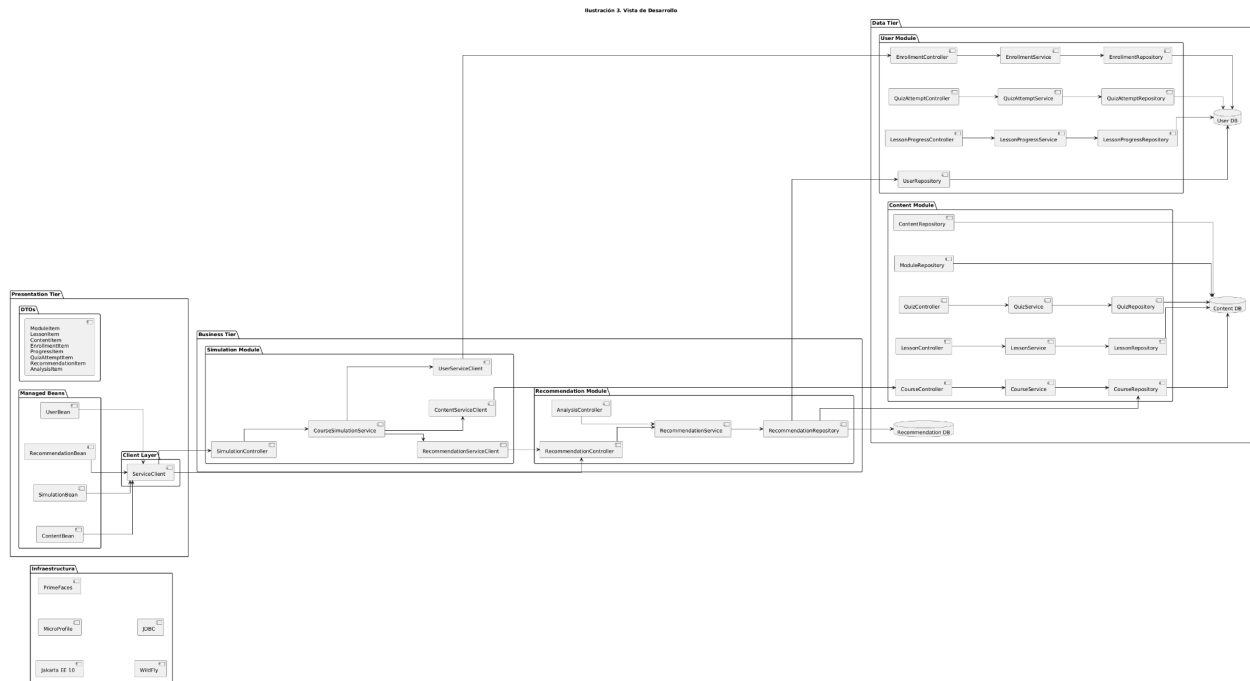
User DB: gestiona la información de los usuarios y su progreso.

Recommendation DB: guarda los resultados generados por el sistema de recomendaciones.

Comunicación entre procesos:

Todos los procesos se comunican mediante servicios REST sobre HTTP/HTTPS, lo que permite una arquitectura desacoplada y escalable. El acceso a los datos se realiza a través de conexiones JDBC hacia las bases de datos correspondientes. Además, el uso de balanceadores de carga garantiza la distribución eficiente de las solicitudes y la tolerancia a fallos.

Vista de Desarrollo



La vista de desarrollo muestra la organización interna del sistema desde la perspectiva de los desarrolladores. La arquitectura de la plataforma E-Learning E.D. está estructurada en una arquitectura multicapa, compuesta por Presentation Tier, Business Tier y Data Tier, apoyada por una capa de infraestructura tecnológica basada en Jakarta EE.

Presentation Tier

La capa de presentación gestiona la interacción con el usuario. Está compuesta por Managed Beans como ContentBean, SimulationBean, RecommendationBean y UserBean, los cuales reciben las solicitudes de la interfaz y se comunican con la capa de negocio a través del ServiceClient.

Además, se utilizan DTOs (Data Transfer Objects) como ModuleItem, LessonItem, ContentItem, EnrollmentItem, ProgressItem y RecommendationItem, que permiten transportar información entre las capas del sistema sin exponer directamente las entidades del modelo de datos.

Business Tier

La capa de negocio implementa la lógica principal del sistema y está organizada en dos módulos: Simulation Module: incluye componentes como SimulationController y CourseSimulationService, que gestionan la simulación de cursos y la interacción con servicios de contenido, usuarios y recomendaciones.

Recommendation Module: incluye RecommendationController, AnalysisController y RecommendationService, encargados de analizar el comportamiento del usuario y generar recomendaciones personalizadas de contenido educativo.

Data Tier

La capa de datos gestiona la persistencia de la información mediante controladores, servicios y repositorios.

Se divide en dos módulos:

Content Module: administra cursos, lecciones, contenidos y evaluaciones mediante componentes como CourseController, LessonController, QuizController y sus respectivos servicios y repositorios.

User Module: gestiona la información de los usuarios, inscripciones, progreso y resultados de evaluaciones mediante EnrollmentController, LessonProgressController y QuizAttemptController.

La información se almacena en tres bases de datos principales:

Content DB: datos de cursos y contenidos educativos.

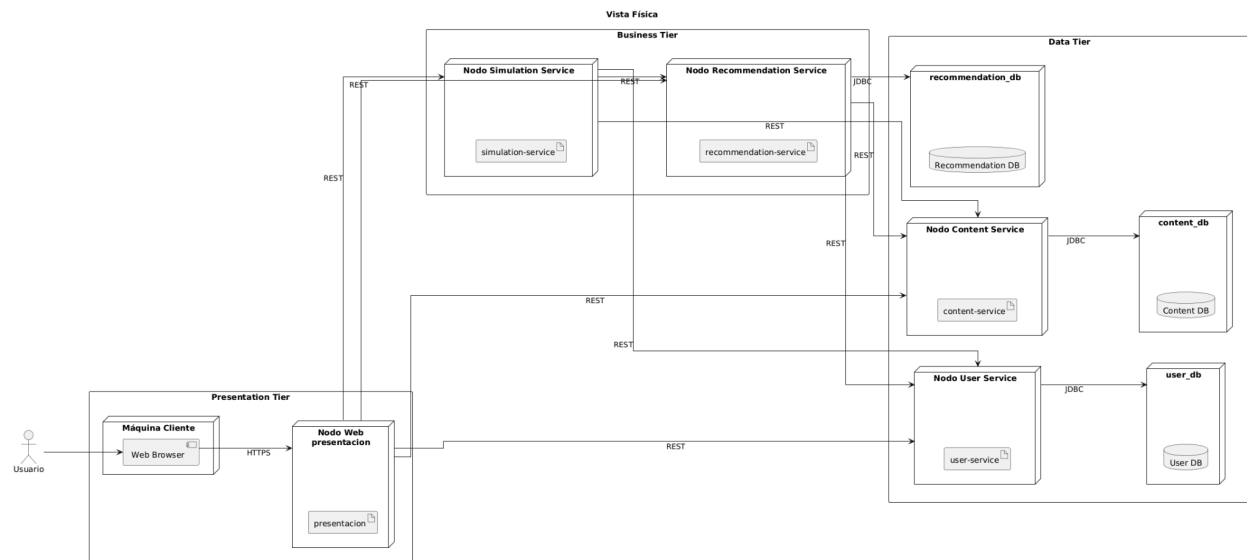
User DB: información de usuarios y progreso de aprendizaje.

Recommendation DB: datos utilizados para el sistema de recomendaciones.

Infraestructura

La plataforma se implementa utilizando Jakarta EE 10, MicroProfile, PrimeFaces para la interfaz web, WildFly como servidor de aplicaciones y JDBC para la conexión con las bases de datos.

Vista Física



El sistema sigue una arquitectura distribuida organizada en tres capas principales: Presentation Tier, Business Tier y Data Tier.

En la Presentation Tier, el usuario accede al sistema a través de un navegador web, el cual se comunica mediante HTTPS con el nodo de presentación que contiene el artefacto presentacion.war. Este componente gestiona la interfaz del sistema y envía solicitudes a los servicios backend.

En la Business Tier se encuentran los servicios encargados de ejecutar la lógica principal del sistema. El simulation-service gestiona la simulación de cursos y el flujo de aprendizaje, mientras que el recommendation-service se encarga de generar recomendaciones personalizadas para los usuarios. Estos servicios se comunican con otros módulos mediante APIs REST.

En la Data Tier se encuentran los servicios responsables de gestionar los datos del sistema. El content-service administra los cursos, lecciones y contenidos educativos, mientras que el user-service gestiona la información de usuarios, inscripciones y progreso de aprendizaje.

Cada servicio se conecta a su propia base de datos utilizando JDBC, lo que permite mantener una separación clara entre los distintos dominios de datos. Las bases de datos incluidas son Content DB, User DB y Recommendation DB.